

One Problem, Three Independent Solutions

Brownian motion with drift — solved by calculus, a grid solver, and 100,000 simulated particles. Validated in SQL.



Calculus



Grid solver



Monte Carlo



A particle that drifts and jitters

Steady push (drift): a breeze carries the particle at speed μ . **Random knocks (diffusion):** unpredictable jostling of strength σ .

So the honest answer is a probability: where is the particle likely to be at time t ?

One particle's rule (SDE): $dX = \mu dt + \sigma dW$

The whole cloud's rule (PDE): $\partial p / \partial t = -\mu \partial p / \partial x + (\sigma^2 / 2) \partial^2 p / \partial x^2$

Why this problem? It has a known exact answer — a perfect calibration case. Any method that reproduces it here can be trusted on problems where nobody knows the answer.



A moving, spreading bell curve

Change to a frame moving with the drift ($y = x - \mu t$): the drift cancels, leaving the classic heat equation — whose solution is a Gaussian (bell curve).

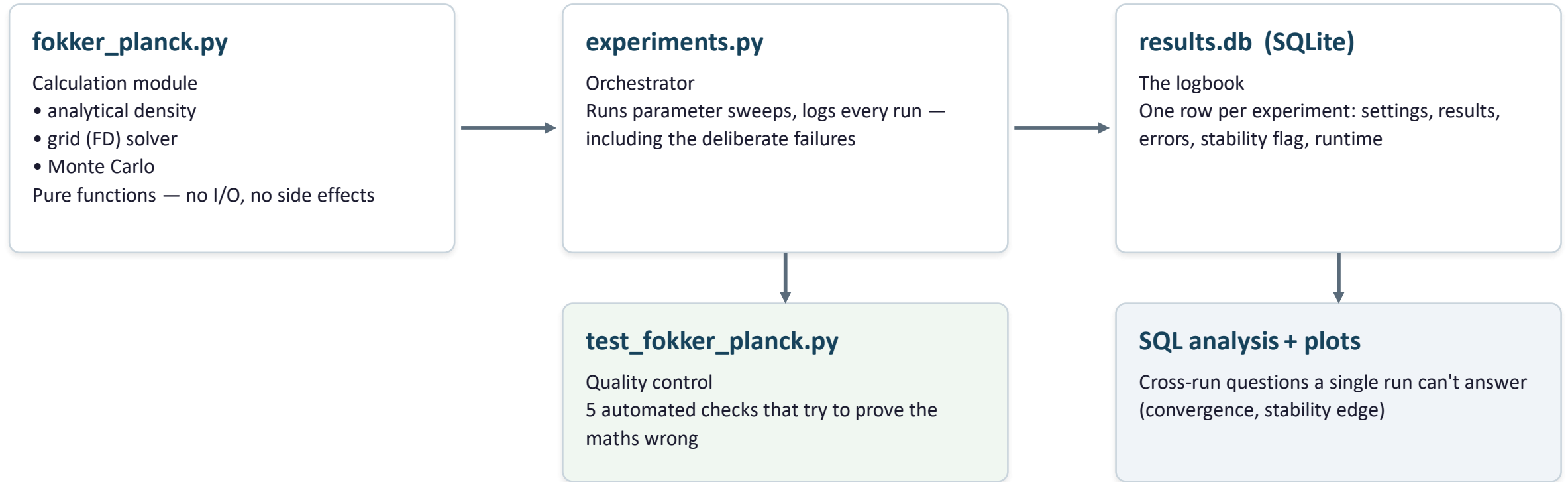
$$p(x, t) = \text{Normal}(\text{mean} = \mu t, \text{variance} = \sigma^2 t)$$

- Centre drifts at μt — the most likely position moves steadily with the breeze.
- Width grows like $\sqrt{t}$ — twice the uncertainty takes four times as long.
- Cross-checked independently: integrating the SDE directly gives the same Gaussian.

This is the ground truth every other method is measured against. [Full derivation → appendix](#)



Compute, persist, verify



Separation of concerns — the maths knows nothing about storage; change one, not both.



Persist everything, incl. failures — the runs that blow up are the most informative rows.



Automate the checking — claims are asserted by tests, not eyeballed from a graph.



Neighbours nudging neighbours

Chop space into buckets, time into ticks. Each tick, every bucket looks at its two neighbours: the drift slides probability along; the diffusion averages it out.

```
p_new[1:-1] = ( p[1:-1]
                - c * (p[2:] - p[:-2])           # slide (drift term)
                + d * (p[2:] - 2*p[1:-1] + p[:-2]) ) # spread (diffusion term)
```

Slide: right neighbour minus left = local slope ($\partial p / \partial x$). **Spread:** neighbours minus twice me = curvature ($\partial^2 p / \partial x^2$) — peaks get shaved, dips get filled.

The catch: Δt and Δx are not independent. Stability requires $r = D\Delta t / \Delta x^2 \leq \frac{1}{2}$ — beyond it, each step amplifies noise instead of damping it.



Be the particle — 100,000 times

The opposite philosophy: never touch the PDE. Simulate one particle's journey — a fixed drift move plus a random knock each step — then repeat 100,000 times and count.

```
increments = mu*dt + sigma*np.sqrt(dt) * rng.standard_normal((n_paths, n_steps))
terminal   = increments.sum(axis=1)      # each particle's final position
```

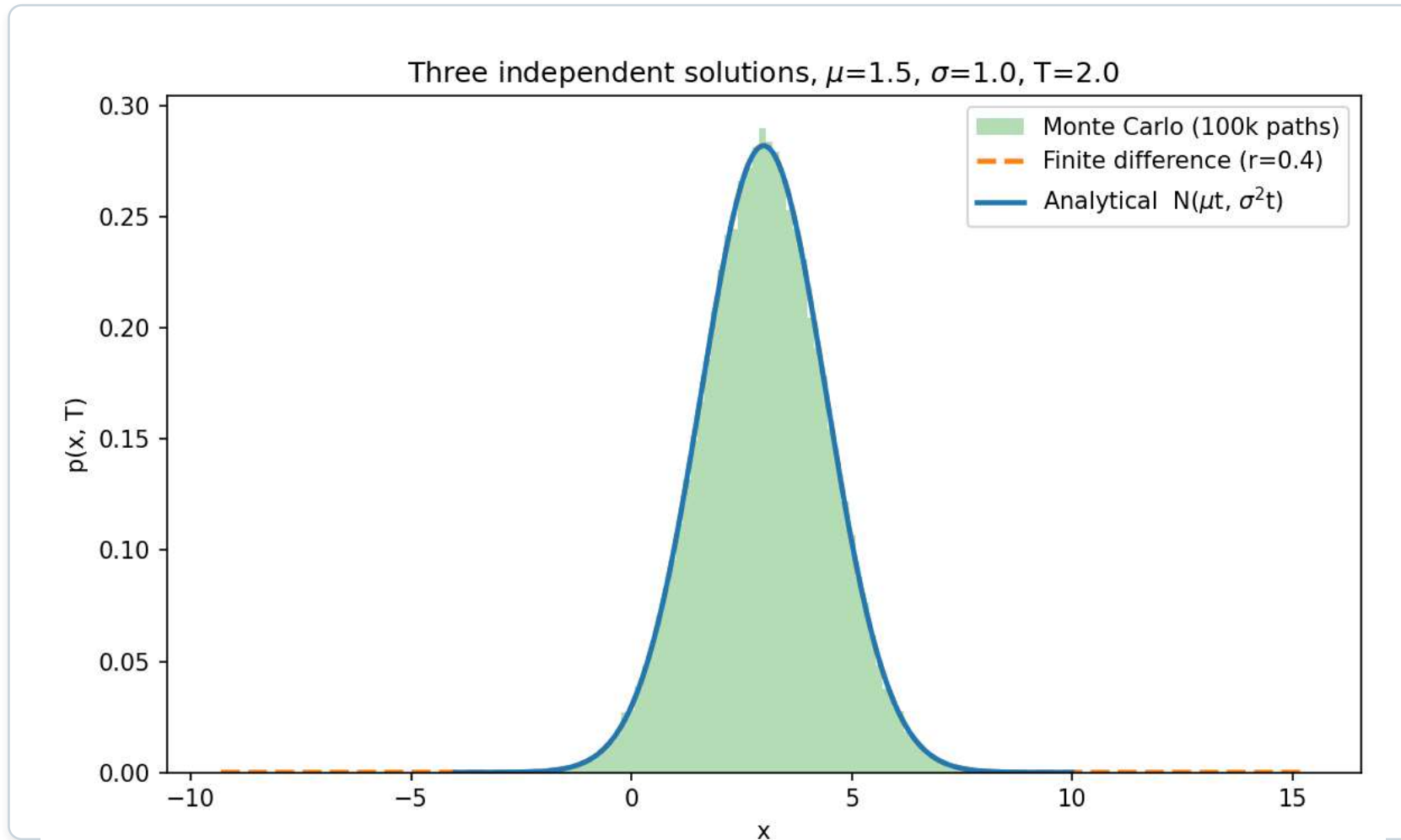
- Fully vectorised — 10 million random movements in two lines, no Python loops.
- Seeded RNG — every result in this talk is exactly reproducible.
- The histogram of final positions IS the answer — no calculus anywhere.

Solver = told the cloud's equation. Monte Carlo = only told one particle's rule.

No shared code, no shared assumptions — exactly what makes agreement meaningful.



Three methods, one curve

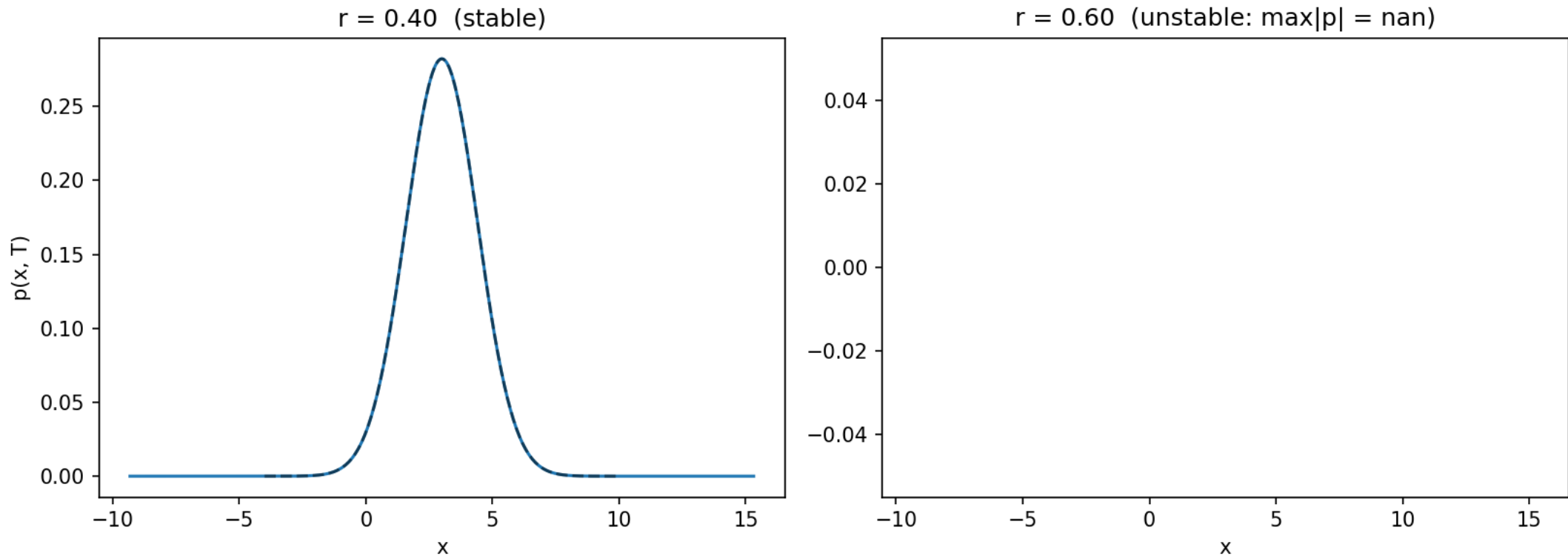


Calculus, grid arithmetic, and dice-rolling land on the same bell curve.



Engineered, not assumed

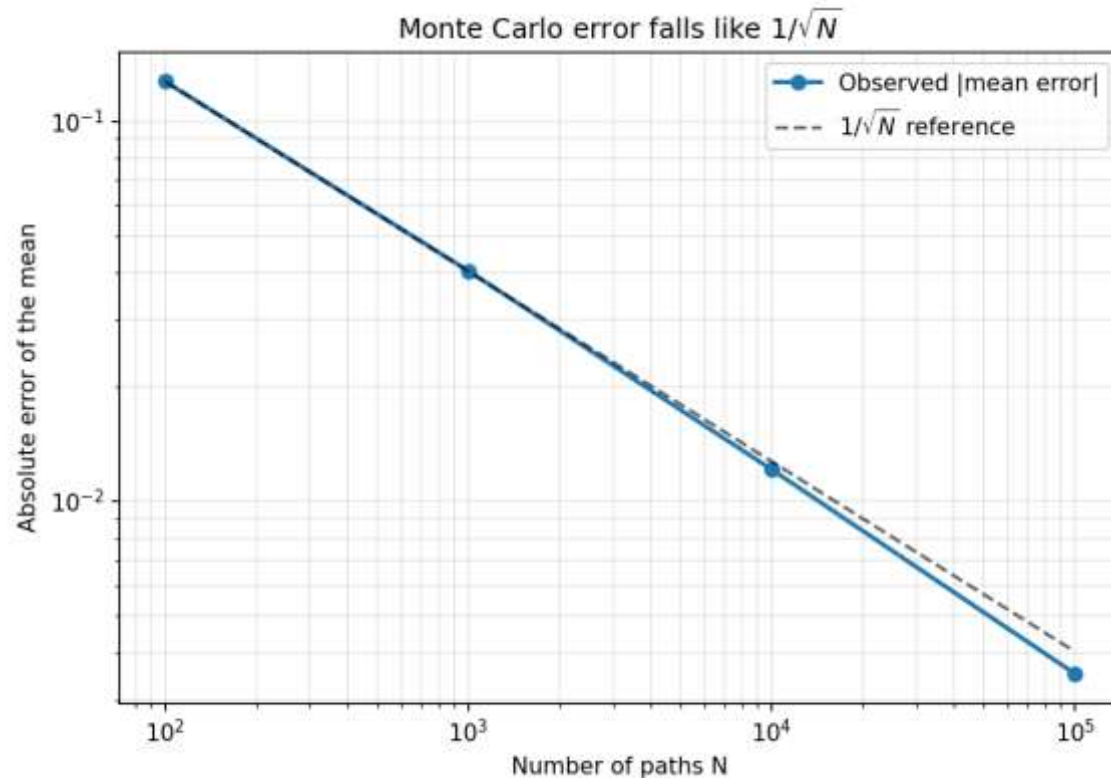
Same scheme, same physics - only the timestep changed (stability: $r = D\Delta t/\Delta x^2 \leq 1/2$)



Same code, same physics — only Δt changed. The failure mode is known, provoked deliberately, and asserted by one of 5 automated tests — alongside mass conservation, exact moments, solver accuracy, and MC convergence.



SQL proves the convergence



```
AVG(err) / LAG(AVG(err))  
OVER (ORDER BY n_paths)
```

A window function compares each batch of runs to the previous batch:

Paths	Avg error	Ratio
100	0.127	—
1,000	0.040	0.317
10,000	0.012	0.30
100,000	0.0035	0.29

Theory predicts $1/\sqrt{10} \approx 0.316$ per tenfold — the database confirms it.



What this demonstrates

- Independent methods agreeing — the strongest evidence an implementation is right.
- Failure modes known, provoked deliberately, and covered by automated tests.
- Every result persisted and queryable — conclusions come from data, not memory.
- Python computes, SQL analyses, tests build trust — structured like production work.

*“Accuracy can’t be taken on faith — it has to be demonstrated.
That’s how I’d approach reliability in your systems too.”*



The derivation in five steps

- 1. Moving frame: $y = x - \mu t$. Chain rule: $\partial p / \partial t = \partial q / \partial \tau - \mu \partial q / \partial y$; space derivatives unchanged.
- 2. Substitute — the drift terms cancel, leaving the pure heat equation: $\partial q / \partial \tau = (\sigma^2 / 2) \partial^2 q / \partial y^2$.
- 3. Fourier transform: derivatives become multiplication $\rightarrow$ ODE per mode: $\hat{q}(k, \tau) = e^{(-\sigma^2 k^2 \tau / 2)}$.
- 4. Invert (Gaussian $\leftrightarrow$ Gaussian): $q(y, \tau) = (2\pi\sigma^2\tau)^{(-1/2)} \cdot e^{(-y^2 / 2\sigma^2\tau)}$.
- 5. Transform back ($y = x - \mu t$): $p(x, t) = (2\pi\sigma^2 t)^{(-1/2)} \cdot \exp(- (x - \mu t)^2 / 2\sigma^2 t)$ — $N(\mu t, \sigma^2 t)$.

Independent check: integrate the SDE directly — $X(t) = \mu t + \sigma W(t)$, and $W(t) \sim N(0, t)$ gives the same result. [← back to talk](#)